

Email Intake

Detecting design changes from sales emails — implemented design and a path to production hardening

1 The method we implemented

A sales email is not a clean API call. It is free text written by a human: it may not request a change at all, it may be vague, it may name a project that does not exist, and, above all, it represents untrusted input. The task is to turn that into a safe, reviewable design change without ever letting an email silently rewrite a project. The design is built around one principle:

*An email can **propose** a change, but only a human can **commit** it.*

The pipeline

The flow is split into four deliberately separate stages rather than one large “read the email and act” call. Each stage has a single responsibility, and the boundary between thinking and writing sits between stage 3 and stage 4.

(1) receive	->	(2) extract	->	(3) propose	->	(4) confirm & commit
webhook		Gemini -> JSON		resolve + run		human clicks Confirm
(read only)		(read only)		(compute only)		(the ONLY write)

Stage 1 — Receive (POST /email/incoming)

A thin webhook accepts {sender, subject, body}. In a real deployment this is what an email provider's inbound-parse webhook (SendGrid, Postmark, Mailgun, or the Gmail API) would POST to. Keeping it a plain HTTP endpoint makes the ingestion source swappable without touching anything downstream.

Stage 2 — Extract

One Gemini call, forced to return structured JSON via a fixed response_schema (project_name, changes[], confidence). This stage only reads; it never calls an engine and never touches the store. Two decisions matter: the output is constrained to a schema (a predictable shape to act on, not a sentence to parse), and the email body is treated as data, never as instructions — the system prompt explicitly forbids following any instruction found inside the email. That is the prompt-injection boundary.

Stage 3 — Propose

This converts the raw extraction into a concrete, validated candidate **without saving**. It resolves the human project name to an id (refusing to guess when the name is ambiguous or unknown), maps loose words such as length onto engine keys such as length_mm, applies a confidence gate so vague emails are treated as not-a-change-request, and runs the engine to compute the new figures and drawing — but does not persist. By calling the exact same store.propose_change() core used by the chat path, validation and drawing generation cannot drift between channels.

Stage 4 — Confirm & commit (the only write)

Actionable proposals are parked under a short random token and surfaced on a review page that shows the requested change, the recomputed key figures, and the regenerated drawing — so the reviewer sees exactly what they are approving. POST /email/confirm/<token> is the only point in the entire flow that writes to the store; /reject discards. Design changes have real downstream cost, so an automated email is never the final authority: the system detects and prepares, and a person decides.

2 Key architectural benefits

- **Strict immutability** — splitting the flow into an explicit pipeline where stages 1–3 are strictly read/compute-only is a deliberate pattern. The system stays deterministic right up to the single execution point, which makes its behaviour easy to reason about and hard to misuse.
- **Prompt-injection security** — treating the email strictly as data and constraining the LLM to a strict JSON schema via `response_schema` is the industry-standard way to mitigate prompt injection. The model only ever performs reading comprehension into a fixed shape; it never has its output trusted as a command.
- **A DRY core** — reusing `store.propose_change()` across both the chat and email channels ensures validation rules and physics never drift between the two. There is one implementation, so the two paths cannot disagree.

3 Where it can be strengthened

The current design is correct, but two production realities sit just outside what the exercise required:

- **Idempotency & retries** — inbound email webhooks (SendGrid, Mailgun) are known to retry a request if the endpoint is slow to respond. Because stage 2 calls the Gemini API — which can take a few seconds — the webhook may time out, prompting the provider to redeliver the same email and produce a duplicate proposal.
- **Race conditions** — if two identical requests reach the in-memory store at the same time, they can each create a separate pending token for what is really one change.

4 The optimisation: an idempotency layer at stage 1

To make the flow bulletproof in production, introduce an idempotency layer right at ingestion and move the slow work off the request path.

```
(1) Webhook ingest ---> check Message-ID in DB ---> enqueue background job ---> 202 Accepted
                                                                |
                                                                v
                                (2) extract ---> (3) propose ---> (4) park
```

Acknowledge immediately

Do not make the webhook wait for the Gemini call. Persist the raw email to a table, check the standard Message-ID header to discard duplicates, and return 202 Accepted to the provider within milliseconds. The Message-ID is a natural idempotency key — a redelivered email carries the same one, so the second arrival is recognised and dropped before any work happens.

Process asynchronously

Let a background worker (Celery, RQ, or a simple task queue) handle stages 2 and 3 out-of-band. This isolates slow, variable LLM latency from HTTP traffic, so webhook response time no longer depends on the model. It also removes the race condition: with the raw email stored under a unique key and a single worker advancing it through the stages, duplicate tokens cannot be created for the same message.

What stays the same

This change is confined to stage 1 and the transport around stages 2–3. The extraction logic, the propose/validate core, the human-confirm gate, and the single-write-point principle are all untouched — the same properties that make the current design safe carry straight into the hardened one.